

ECE 457A-Adaptive and Cooperative Algorithms

Population Based Algorithms

University of Waterloo

June 12, 2026

Search Algorithms: Assumptions

Basic fact

- ▶ A completely blind search behaves like random search.
- ▶ To do better, an algorithm must use **some assumption** about the problem.

Examples of assumptions used by different algorithms

Classical / model-based methods

- ▶ Steepest Ascent: nearby solutions improve gradually
- ▶ Newton–Raphson: derivatives exist and are meaningful
- ▶ Levenberg–Marquardt: cost surface is locally quadratic
- ▶ Exhaustive Search: problem size is small enough

Metaheuristic methods

- ▶ Tabu Search: memory can prevent cycling
- ▶ Simulated Annealing: occasional worsening helps exploration
- ▶ PSO: good regions attract the population
- ▶ Evolutionary Algorithms: good structures survive and combine

Key message

- ▶ No algorithm is assumption-free.
- ▶ Algorithms differ in *what* they assume and *how strongly*.

Population Based: Why another optimization paradigm?

Many engineering problems share the following properties:

- ▶ Non-convex, multimodal objective functions
- ▶ Discontinuous or noisy evaluations
- ▶ No closed-form gradients
- ▶ Multiple competing design objectives

Takeaway

These properties fundamentally limit classical trajectory-based optimization.

Insight

This slide frames GA as a response to problem structure, not as a replacement for calculus.

Trajectory-based search: intuition

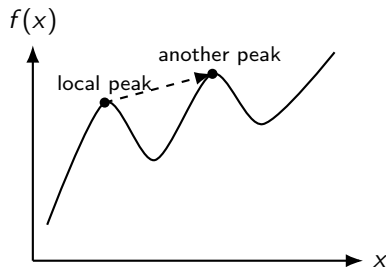
Trajectory methods:

- ▶ Maintain a **single candidate solution**

- ▶ Generate a sequence:

$$x^{(0)} \rightarrow x^{(1)} \rightarrow x^{(2)} \rightarrow \dots$$

- ▶ Use local information (gradient, neighborhood, acceptance rule)



Takeaway

Trajectory methods improve *locally*; escaping poor basins is difficult.

Population-based search

- ▶ Maintain a **set of candidate solutions**

$$P^{(t)} = \{x_1^{(t)}, x_2^{(t)}, \dots, x_N^{(t)}\}$$

- ▶ Explore *multiple regions* of the search space simultaneously
- ▶ Allow interaction between solutions

Takeaway

Population = built-in parallelism and diversity.

Population-Based Algorithms Families

- ▶ **Population-based algorithms** address hard search problems by:
 - ▶ maintaining **multiple candidate solutions** in parallel,
 - ▶ preserving **diversity** to avoid premature convergence,
 - ▶ balancing **exploration** and **exploitation**.

Key idea

Search is driven by the **collective behavior** of a population, not a single trajectory.

Major families

- ▶ **Evolutionary Algorithms (EA)**
 - ▶ Genetic Algorithms (GA)
 - ▶ Evolution Strategies (ES)
 - ▶ Evolutionary Programming (EP)
- ▶ **Swarm Intelligence**
 - ▶ Particle Swarm Optimization (PSO)
 - ▶ Ant Colony Optimization (ACO)

Takeaway

Population vs trajectory

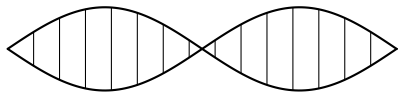
Aspect	Trajectory	Population
State	One solution	Many solutions
Exploration	Limited	Inherent
Parallelism	None	Natural
Diversity	Not explicit	Explicitly managed
Escape local optima	Difficult	Easier

Takeaway

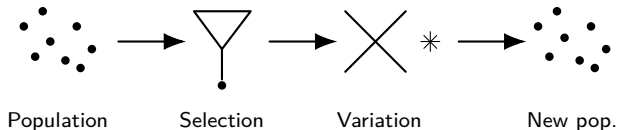
Population-based search trades computation for robustness.

Evolutionary Computing

Biologically Inspired Problem Solving



Nature: *variation + selection*



Context

- ▶ Many real problems have **vast** search spaces and **no reliable gradients** (discrete, noisy, multi-modal).
- ▶ We keep a **population** of candidate solutions and improve them by repeating: **evaluate** → **select** → **vary** (recombine/mutate).
- ▶ Core design questions: **representation**, **fitness**, **operators**, and **diversity vs convergence**.

Evolutionary computing

Evolutionary algorithms abstract the following principles:

- ▶ Variation (generate novelty)
- ▶ Selection (bias toward quality)
- ▶ Competition for limited resources: Individuals cannot all survive or reproduce. Fitness determines access to scarce resources such as selection slots, mating opportunities, or population space. Competition ensures that improvements are *relative*, not absolute; progress is driven by outperforming others.
- ▶ Iterative adaptation

They **do not** attempt to simulate biology accurately.

Takeaway

Evolution is a *computational metaphor*, not a biological model.

Adaptation

Adaptation occurs at multiple levels

- ▶ New behaviors and organs
- ▶ New cells and cellular behaviors
- ▶ New proteins
- ▶ New genes

Key takeaway

Adaptation is a **multi-level optimization process**.

Evolutionary Computing: Core Ideas

What evolutionary algorithms are

- ▶ **Stochastic**: search decisions involve randomness
- ▶ **Population-based**: multiple candidate solutions evolve in parallel
- ▶ **Generate-and-test**: new solutions are generated and evaluated via fitness

Forces driving evolution

- ▶ **Variation** (recombination, mutation)
 - ▶ introduces novelty
 - ▶ enables exploration of new regions
- ▶ **Selection**
 - ▶ filters solutions based on fitness
 - ▶ amplifies high-quality structures

Genetic Algorithms

Population-Based Search via Evolutionary Principles

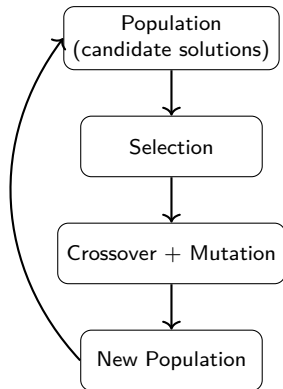
Context

Genetic Algorithms are a foundational class of **population-based stochastic optimization methods** inspired by natural evolution.

Why Genetic Algorithms?

- ▶ Among population-based methods, **Genetic Algorithms** are:
 - ▶ conceptually simple,
 - ▶ widely applicable,
 - ▶ strongly inspired by natural evolution.
- ▶ GA operates on a population using:
 - ▶ **selection** (survival of the fittest),
 - ▶ **recombination** (information mixing),
 - ▶ **mutation** (innovation).

Lecture focus: GA fundamentals, then comparison with PSO, ACO, and cooperative variants.



Population evolves through repeated selection and variation

No Free Lunch & Active Information: Intuition

- ▶ **No Free Lunch (NFL):** Averaged over all possible objective functions, *no search algorithm outperforms random search*.
- ▶ Therefore, whenever an algorithm performs well, it must be exploiting **problem-specific structure**.
- ▶ In Genetic Algorithms, this structure is injected through:
 - ▶ the **representation** (what patterns can exist),
 - ▶ **variation operators** (what patterns are preserved or disrupted),
 - ▶ **selection** (what information is retained).

Takeaway

GA performance comes from **matching representation and operators** to the structure of the fitness landscape.

Insight

Recurring course message: Algorithms do not create intelligence — they encode assumptions. Always ask: "What assumptions does this operator impose on the problem?"

Mapping biology to optimization

Biology	GA	Optimization meaning
Individual	Candidate solution	Design point
Gene	Variable	Parameter
Chromosome	Encoding	Representation
Fitness	Fitness function	Objective value
Environment	Problem	Constraints + objective

Takeaway

Every GA design choice corresponds to an assumption about the problem.

The Notion of Fitness

Fitness

- ▶ Assigns a scalar quality measure to each point in the search space Ω
- ▶ Reflects how well a solution satisfies problem objectives and constraints

Role in search

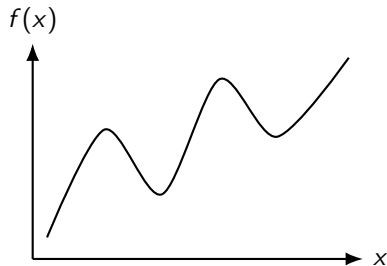
- ▶ Guides the search toward **acceptable or high-quality solutions**
- ▶ Is the **only feedback signal** available to many algorithms
- ▶ Is entirely defined by the **environment** (problem formulation)

Geometric interpretation

Target region $T \subset \Omega$ (solutions with acceptable fitness)

Fitness landscape viewpoint

- ▶ Every candidate solution corresponds to a point in parameter space
- ▶ The fitness function induces a **landscape**
- ▶ Optimization = navigating this landscape efficiently

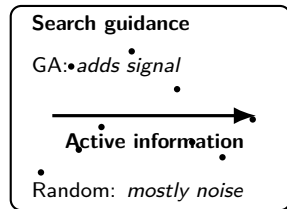


Takeaway

GA searches the landscape *implicitly*, without gradients.

Active information (GA Context)

- ▶ Random search uses no problem-specific information.
- ▶ Any improvement over random search requires **active information**.
- ▶ In GAs, this information enters via:
 - ▶ Representation
 - ▶ Crossover
 - ▶ Mutation
 - ▶ Selection pressure



Takeaway

GA success depends on how well these components encode structure.

Representation: where problem structure is encoded

- ▶ Representation is the **language** of the search.
- ▶ A good encoding makes **small genetic changes** correspond to **meaningful solution changes**.
- ▶ A bad encoding breaks structure: neighbors in genotype become unrelated in phenotype.

Mapping quality

Good: smooth changes



small genetic change $\Rightarrow$ small behavioral change

Bad: chaotic jumps



neighboring genotypes $\Rightarrow$ unrelated solutions

Takeaway

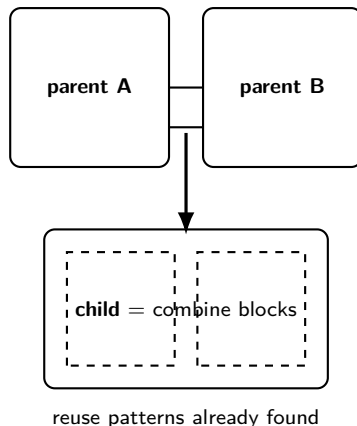
Representation controls whether search steps are meaningful or chaotic.

Crossover: Reusing useful building blocks

- ▶ Crossover is a **structured guess**:
 - ▶ “good partial solutions can be combined.”
- ▶ It is most effective when the encoding preserves **building blocks**.
- ▶ Poorly aligned crossover can **destroy** structure faster than it creates it.

Takeaway

Crossover exploits what the population already learned.

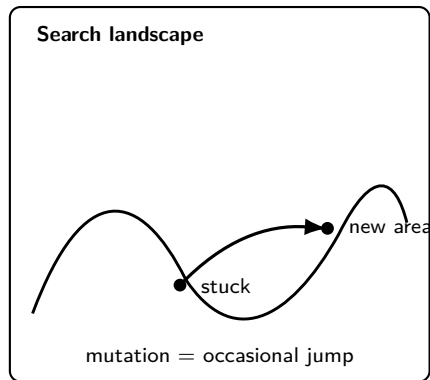


Mutation: Keeping exploration alive

- ▶ Mutation provides **fresh directions** when the population becomes similar.
- ▶ It prevents the search from getting stuck in **local traps**.
- ▶ Think of it as a small **random nudge** that can open new pathways.

Takeaway

Mutation injects novelty; it is the “escape hatch.”

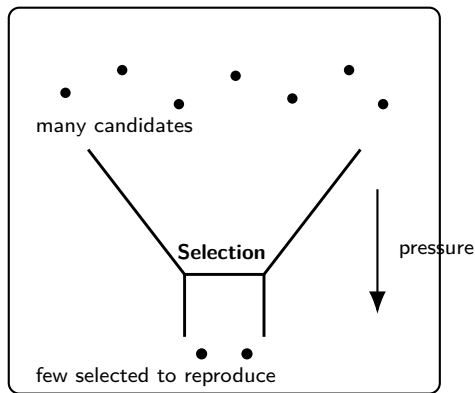


Selection pressure

- ▶ Selection turns **fitness differences** into **reproduction chances**.
- ▶ Too weak: slow learning (nearly random drift).
- ▶ Too strong: premature convergence (loss of diversity).

Takeaway

Selection pressure controls the exploration–exploitation balance.



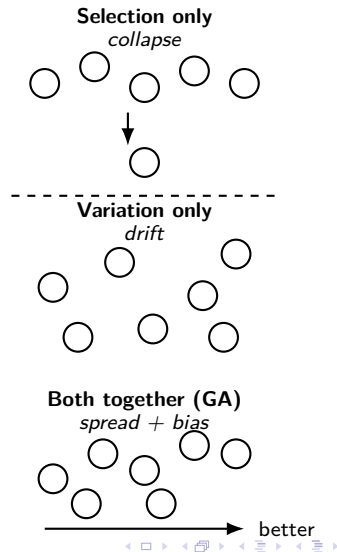
Selection vs. Variation

- ▶ Selection without variation collapses diversity
- ▶ Variation without selection is random search

$$\text{GA} = \text{Variation} + \text{Selection}$$

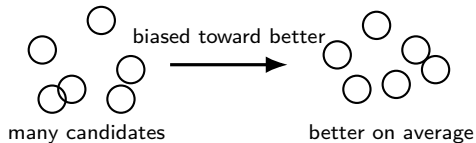
Takeaway

The balance between exploration and exploitation is central.



Simple Genetic Algorithm (SGA)

The canonical baseline GA



We use SGA to highlight the core principles: representation, variation, and selection.

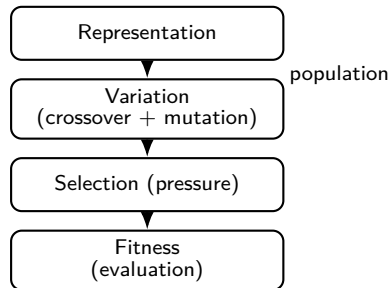
What is a Simple Genetic Algorithm?

The **Simple Genetic Algorithm (SGA)** is the canonical GA formulation:

- ▶ Proposed by **John Holland (1975)** as a population-based adaptation mechanism.
- ▶ Serves as a **baseline** for understanding (and comparing) GA variants.
- ▶ Uses **fixed** operators and parameters (no adaptive schedules, no fancy hybrids).

What it deliberately assumes (to stay “simple”):

- ▶ a single representation (often fixed-length strings)
- ▶ a single fitness function to rank candidates,
- ▶ standard crossover + mutation,
- ▶ selection with a constant pressure.



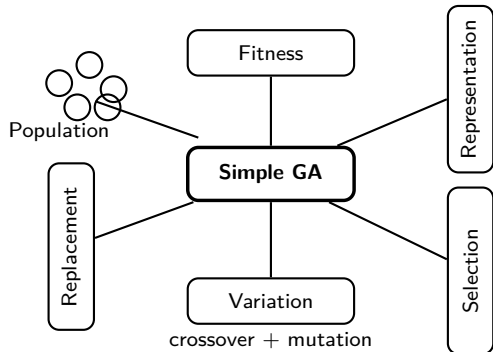
Insight

Nearly every modern GA can be viewed as a modification of one SGA component.

Core ingredients of SGA

An SGA consists of:

- ▶ A **population** of candidate solutions
- ▶ A **fitness function**
- ▶ A **representation** (encoding)
- ▶ **Selection** mechanisms
- ▶ **Variation operators**
 - ▶ Crossover
 - ▶ Mutation
- ▶ A **replacement** strategy



Takeaway

Every GA design choice or strategy lives inside one of these components.

SGA: high-level algorithm

1. Initialize a population of candidate solutions
2. Evaluate fitness of each individual
3. Repeat until termination:
 - 3.1 Select parents
 - 3.2 Apply crossover
 - 3.3 Apply mutation
 - 3.4 Select survivors

Takeaway

SGA alternates between *evaluation* and *population transformation*.

SGA algorithm (pseudo-code)

Simple Genetic Algorithm

1. Initialize population $P^{(0)}$
2. Evaluate fitness $f(x)$ for all $x \in P^{(0)}$
3. $t \leftarrow 0$
4. **while** termination condition not met:
 - 4.1 Select parents from $P^{(t)}$
 - 4.2 Apply crossover to parents
 - 4.3 Apply mutation to offspring
 - 4.4 Form new population $P^{(t+1)}$
 - 4.5 Evaluate $P^{(t+1)}$
 - 4.6 $t \leftarrow t + 1$

Insight

Replacement policy matters enormously.

Population initialization

Typical initialization strategies:

- ▶ Uniform random sampling
- ▶ Random but feasible (constraint-aware)
- ▶ Seeding with heuristic solutions

Initialization affects:

- ▶ Early diversity
- ▶ Convergence speed
- ▶ Risk of bias

Takeaway

Initialization shapes the first-generation search bias.

Fitness functions

- ▶ Fitness is the *only feedback* the GA receives
- ▶ Can represent:
 - ▶ Objective value
 - ▶ Penalized objective (constraints)
 - ▶ Ranking or dominance score

Key requirement:

- ▶ Fitness must be **comparable across individuals**

Takeaway

If fitness is poorly designed, no GA operator can fix it.

Parent selection vs survivor selection

There are **two distinct selection events** in SGA:

- ▶ **Parent selection**: who reproduces?
- ▶ **Survivor selection**: who remains?

They serve different purposes:

- ▶ Parent selection controls **selection pressure**
- ▶ Survivor selection controls **diversity retention**

Takeaway

Mixing exploration (variation) with exploitation (selection) destabilizes the search by collapsing diversity too early.

SGA: A hands-on example (setup)

Toy optimization problem:

- ▶ Search space: binary strings of length 6
- ▶ Objective: maximize the number of 1s

Representation:

$$x = b_1 b_2 b_3 b_4 b_5 b_6, \quad b_i \in \{0, 1\}$$

Fitness function:

$$f(x) = \text{number of 1s in } x$$

Takeaway

The example is simple on purpose — to isolate SGA mechanics.

Step 2: Evaluate fitness

Compute fitness for each individual:

Individual	String	$f(x)$
x_1	1 0 1 0 0 1	3
x_2	0 1 0 1 1 0	3
x_3	1 1 0 0 0 0	2
x_4	0 0 1 1 0 1	3

The initial population consists of four individuals $\{x_1, x_2, x_3, x_4\}$ sampled at initialization.

Fitness introduces a **ranking bias** into the population.

Takeaway

Fitness converts problem structure into selection pressure.

Step 3: Selection

Using fitness-proportional selection:

- ▶ Higher fitness $\Rightarrow$ higher chance of reproduction
- ▶ Low-fitness individuals may still survive

Assume selected parents:

$$(x_1, x_2), \quad (x_4, x_1)$$

Step 4: Crossover

mix the two solutions (will learn strategies later):

$$x_1 = 101 \mid 001$$

$$x_2 = 010 \mid 110$$

Offspring:

$$y_1 = 101110$$

$$y_2 = 010001$$

Takeaway

Rcombining existing building blocks.

Step 5: Mutation

Apply random bit-flip mutation based on specified probability:

$$y_1 = 101110 \Rightarrow 101\mathbf{0}10$$

Mutation:

- ▶ introduces new genetic material
- ▶ prevents population collapse

Step 6: Replacement

Form the next generation by replacing some or all parents:

Individual	String	$f(x)$
y_1	1 0 1 0 1 0	3
y_2	0 1 0 0 0 1	2
x_2	0 1 0 1 1 0	3
x_4	0 0 1 1 0 1	3

This completes **one SGA generation**.

Takeaway

Evolution = repeat evaluation, selection, variation, replacement.

On Selection & Selection-Pressure (Concepts)

Selection (what it does)

- ▶ The mechanism that chooses **parents** (and sometimes survivors)
- ▶ Biases reproduction toward better solutions while still allowing chance

Common selection schemes

- ▶ **Fitness-proportionate** (roulette wheel):
 $p_i \propto f_i$
- ▶ **Tournament** selection: best-of- k
- ▶ **Rank-based** selection: probability depends on rank (not raw fitness)

Why selection matters

Selection is the **driving force** that converts fitness differences into **reproductive advantage**.

Selection pressure (how strong it is)

Definition

How strongly selection favors **high-fitness** individuals over others.

High pressure:

- ▶ Fast convergence
- ▶ Diversity collapses quickly
- ▶ Higher risk of premature convergence

Low pressure:

- ▶ Slower convergence
- ▶ Better exploration/diversity
- ▶ More robust to local optima

What increases pressure?

Smaller parent pool, larger tournament k , strong elitism, or highly skewed fitness scaling.

Selection
defines *who reproduces*;
Pressure
defines *how aggressively* the best take over.

Elitism: Preserving the best

Elitism ensures that the best solution found so far is not lost.

Common strategies:

- ▶ Keep 1 best individual
- ▶ Replace worst individual with best-so-far

Risk:

- ▶ Too much elitism $\Rightarrow$ premature convergence

Takeaway

Elitism stabilizes progress but amplifies selection pressure.

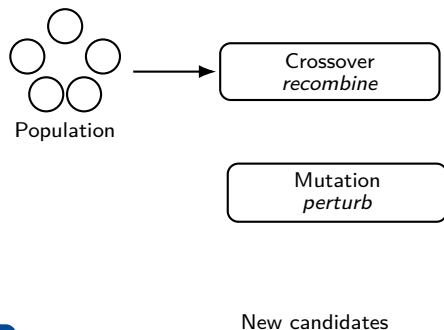
Variation operators: overview

SGA uses two variation operators:

- ▶ **Crossover** (recombination)
- ▶ **Mutation** (random alteration)

Their roles are complementary:

- ▶ Crossover: combine **existing building blocks**
- ▶ Mutation: inject **novelty and diversity**



Takeaway

Variation creates candidates; selection decides their fate.

Termination criteria

Typical termination conditions:

- ▶ Maximum number of generations
- ▶ Maximum number of fitness evaluations
- ▶ Convergence of population
- ▶ Target fitness reached

Important distinction:

- ▶ Termination does **not** imply optimality

Takeaway

GA is an anytime algorithm — stop when resources run out.

Open Matters

SGA intentionally leaves open:

- ▶ Representation
- ▶ Selection scheme
- ▶ Crossover type
- ▶ Mutation rate
- ▶ Population size

These choices determine:

- ▶ Search behavior
- ▶ Convergence speed
- ▶ Robustness

Summary

:

- ▶ The SGA algorithmic loop
- ▶ The role of each component
- ▶ The distinction between selection and replacement

Next lecture:

- ▶ Representation (binary, genotype vs phenotype)
- ▶ Crossover and mutation in detail
- ▶ Exploration vs exploitation

Takeaway

Understanding SGA mechanics is essential before tuning or extending GAs.

Generational Genetic Algorithms

Population-wide replacement dynamics

Core idea

At each generation, the **entire population** is replaced by offspring produced via selection, crossover, and mutation.

Generational Genetic Algorithm (GGA)

Generational GA is the **classical** form of genetic algorithm and serves as the **baseline model** in most textbooks.

Core evolutionary cycle

Selection → Recombination → Mutation → **Full population replacement**

Key idea

- ▶ Evolution proceeds in **discrete generations**
- ▶ Parents are discarded after producing offspring
- ▶ The next generation is composed **entirely of new individuals**

Relation to the Simple GA (SGA):

- ▶ The **Simple Genetic Algorithm (SGA)** is a *specific instantiation* of a Generational GA.
- ▶ It assumes fixed-length encoding, fitness-proportional selection, standard crossover, mutation, and **full replacement**.
- ▶ **GGA** is the broader class; **SGA** is the textbook “canonical example.”

Takeaway

GGA is the **simplest population-wide evolutionary model**.

Steady-State Genetic Algorithm (SSGA)

Steady-State GA is an alternative evolutionary model where **population replacement is incremental rather than generational**.

Core evolutionary cycle

Selection → Recombination → Mutation → **Partial population replacement**

Key idea

- ▶ Evolution proceeds **continuously**, not in full generations
- ▶ Only a small number of individuals are replaced at each step
- ▶ High-fitness individuals may persist for many iterations

Why “steady-state”?

The population remains in a **near-steady configuration**, with only small changes at each iteration instead of full replacement.

Takeaway

SSGA emphasizes **stability and diversity retention** over speed.

Incremental replacement: At each iteration, only a few offspring are introduced and a few individuals are removed; the population evolves *continuously* rather than being fully replaced.

Generational vs Steady-State Genetic Algorithms

Generational GA (GGA)

- ▶ Entire population replaced each generation
- ▶ Each individual lives exactly one generation

Characteristics

- ▶ Simple to implement
- ▶ Strong selection pressure
- ▶ Risk of losing good solutions

Takeaway

Conceptually clean, but usually needs **elitism**.

Steady-State GA (SSGA)

- ▶ Only a few individuals replaced at a time
- ▶ Population evolves gradually

Characteristics

- ▶ Lower selection pressure
- ▶ Better diversity retention
- ▶ More complex bookkeeping

Takeaway

Trades simplicity for smoother, more stable evolution.